Healthcare App Backend: A Beginner's Guide

Oct 16, 2026 · @kratika agrawal

1. The big picture

This app lets a logged-in user keep a list of patients and doctors, and record which doctors look after which patient. This guide explains every idea behind it, starting from zero, with most of the focus on the backend and its data models.

How to read this guide: go top to bottom the first time. Each section builds on the one before. Keep the project open next to you and find each file as it is mentioned. Words in **bold** are defined in the glossary at the end.

Think of a small clinic

In a clinic	In our app
The waiting room where you fill a form	The **frontend** (React website in the browser)
The receptionist who checks your form and fetches files	The **backend** (Node.js + Express server)
The filing cabinet with all the records	The **database** (MongoDB)
Your visitor badge, shown at every door	The **access token** (JWT)
A note linking a patient's file to a doctor's file	A **mapping** (assignment) record

What a user can do

- **Register** and **log in**.
- **Patients:** add, list, view, edit, delete. Each user sees only the patients they added.
- **Doctors:** add, list, view, edit, delete. Everyone sees all doctors, but only the person who added a doctor can edit or delete it.
- **Assignments:** link a doctor to a patient. One patient can have many doctors, and one doctor can have many patients.

2. Frontend, backend, database

Every web app like this one has three parts, and they only talk to the neighbour next to them: the browser never touches the database directly.

- **Frontend** (`frontend/` folder): the screens the user sees, built with React and run by Vite at `http://localhost:5173`. It shows forms and tables, and sends requests to the backend.
- **Backend** (`backend/` folder): a Node.js program using Express, running at `http://localhost:5001`. It receives requests, checks who is asking, checks the input, reads or writes the database, and sends back an answer.
- **Database** (MongoDB, for example on MongoDB Atlas in the cloud): stores the data permanently, so it is still there after the server restarts.

Why not let the browser talk to the database directly? Because anyone can open the browser's developer tools and change the code running there. The backend is the trusted middle person: it decides what each user is allowed to see and change.

During development: the Vite proxy

The frontend calls URLs like `/api/patients`. Vite (the frontend dev server) forwards anything starting with `/api` to `http://localhost:5001`. This setting lives in `frontend/vite.config.js`. Because of it, the browser thinks it is talking to one website, which avoids CORS problems (section 14).

3. Requests and responses (HTTP)

The frontend and backend talk by sending messages called **requests** and **responses**, using a set of rules called **HTTP**. It works like a letter and its reply.

A request has four parts:

1. **Method:** what you want to do.
2. **URL:** what you want to do it to, e.g. `/api/patients/65f1...`.
3. **Headers:** extra information, e.g. `Authorization: Bearer <token>` (who you are) and `Content-Type: application/json` (what format the body is in).
4. **Body** (optional): the data you send, written as **JSON**.

The methods we use

Method	Meaning	Example in our app
GET	Read something	`GET /api/patients` lists your patients

Method	Meaning	Example in our app
POST	Create something	`POST /api/doctors` adds a doctor
PUT	Update something	`PUT /api/patients/:id` edits a patient
DELETE	Remove something	`DELETE /api/mappings/:id` removes an assignment

`:id` is a placeholder. In a real request it is replaced by an actual id, and the backend reads it as `req.params.id`.

JSON

JSON is a text format for data, made of keys and values:

```
{ "name": "Asha", "age": 34, "gender": "female" }
```

Status codes: the number on the reply

Every response carries a number that says how things went.

Code	Meaning	When our app sends it
200	OK	A read or update worked
201	Created	Something new was saved
400	Bad request	The input was wrong, e.g. age is "abc"
401	Unauthorized	Not logged in, or the token expired
403	Forbidden	Logged in, but not allowed (editing someone else's doctor)
404	Not found	That record doesn't exist, or isn't yours
409	Conflict	Duplicate, e.g. that email is already registered
500	Server error	A bug or a crash on the backend

Rule of thumb: 2xx = success, 4xx = the client did something wrong, 5xx = the server did something wrong.

The shape of our responses

Every response from our API has `success: true` or `success: false`:

```
{ "success": true, "message": "Patient created", "patient": { "id":
"65f1...", "name": "Asha" } }
{ "success": false, "message": "Validation failed", "errors": [{ "field":
"age", "message": "Age must be between 0 and 150" }] }
```

4. A tour of the backend folder

The backend code is split into small folders, each with one job. When you want to change something, this table tells you where to look.

File or folder	Its one job
`src/server.js`	Starts everything: connects to the database, then starts listening for requests
`src/app.js`	Builds the Express app: turns on CORS, JSON reading and logging, then plugs in the routes
`src/config/env.js`	Reads settings from the `.env` file (port, database link, secrets)
`src/config/db.js`	Connects to MongoDB
`src/models/`	Describes the shape of our data: User, Patient, Doctor, Mapping
`src/routes/`	The menu: which URL + method goes to which function
`src/validators/`	Rules for checking input (is the email valid? is age a number?)
`src/middleware/auth.js`	Checks the access token and finds the logged-in user
`src/middleware/validate.js`	Stops the request if the validators found problems
`src/middleware/errorHandler.js`	Turns any error into a clean JSON reply
`src/controllers/`	The real work for each action: talk to the database, build the reply
`src/utils/ApiError.js`	A small helper to create errors with a status code
`src/utils/tokens.js`	Creates the access and refresh tokens
`.env`	Your secret settings. Never commit this file to Git

File or folder	Its one job
`postman_collection.json`	Ready-made requests to test the API in Postman

This split is often called **MVC-style** (Model, View, Controller). Here the "view" is just the JSON we send back; the React app draws the actual screens.

5. The journey of one request

A request passes through a chain of small functions, one after another, like a file passed between desks. If any desk finds a problem, it stops the chain and the error handler sends the reply.

A request stops at the first failed check



request journey · 7 steps, 3 ways out to the error handler

Only a request that passes every check reaches the controller; any failure jumps straight to the error handler.

Example: creating a patient

The frontend sends `POST /api/patients` with a token in the header and the patient in the body. Here is what happens, in order:

1. **`app.js`** – `express.json()` reads the JSON body and puts it in `req.body`. `cors()` and `morgan()` (logging) also run here.
2. **`app.js`** – the URL starts with `/api/patients`, so the request goes to `routes/patientRoutes.js`.
3. **`auth` middleware** – reads the token, checks it, loads the user from the database and saves it as `req.user`. No valid token → 401, stop.
4. **Validators** – check `name`, `age`, `gender` and so on.
5. **`validate` middleware** – if any check failed → 400 with the list of problems, stop.
6. **Controller** `patientController.create` – saves the patient in MongoDB with `createdBy: req.user._id`.
7. **Response** – `201` with `{ success: true, message: 'Patient created', patient }`.

What these words mean

- **Express:** a library that makes it easy to write a web server in Node.js.
- **Route:** a pair of method + URL linked to functions, e.g. `router.post('/', ...)`.
- **Middleware:** a function that runs before the controller. It gets `(req, res, next)`. It either calls `next()` to pass the request on, or stops it (by sending a reply or throwing an error).
- **Controller:** the last function in the chain. It does the real work and sends the reply.
- **`req`** (request): everything the client sent: `req.body`, `req.params`, `req.headers`, plus `req.user` that our auth middleware adds.
- **`res`** (response): what we use to reply, e.g. `res.status(201).json({...})`.

The route file, line by line

```
router.use(auth); // every route below needs a logged-in user
router.route('/')
  .post(v.patient(), validate, ctrl.create) // POST /api/patients
  .get(ctrl.list);                          // GET  /api/patients
router.route('/:id')
  .get(v.idParam(), validate, ctrl.get)
  .put(v.idParam(), v.patient(true), validate, ctrl.update)
  .delete(v.idParam(), validate, ctrl.remove);
```

Read each line left to right: that is the order the functions run in.

Async code and errors

Talking to the database takes time, so controllers are `async` functions and use `await`, e.g. `await Patient.create(...)`. When something goes wrong, a controller just does `throw ApiError.notFound('Patient not found')`. We use **Express 5**, which catches errors thrown in `async` functions and passes them to the error handler automatically. (Express 4 could not do this, which is why older projects wrap every route in an `asyncHandler` helper. See section 15.)

6. MongoDB and Mongoose basics

MongoDB stores data as **documents** (JSON-like objects) grouped into **collections**. Mongoose is a library that lets our Node.js code describe and use those documents easily.

MongoDB words, compared with a spreadsheet

MongoDB	Spreadsheet idea	In our app
Database	The whole file	`healthcare_db`
Collection	One sheet	`users`, `patients`, `doctors`, `patient_doctor_mappings`
Document	One row	One patient
Field	One column	`name`, `age`

A patient document looks like this inside MongoDB:

```
{
  "_id": "65f1c2a9e4b0a1b2c3d4e5f6",
  "name": "Asha",
  "age": 34,
  "gender": "female",
  "createdBy": "65f1c0aa11b0a1b2c3d4e5aa",
  "createdAt": "2026-10-16T08:00:00.000Z",
  "updatedAt": "2026-10-16T08:00:00.000Z",
  "__v": 0
}
```

- **`_id`**: every document gets a unique id automatically. Its type is **ObjectId**: 24 characters of 0–9 and a–f.
- **`createdAt`** / **`updatedAt`**: added because our schemas use `{ timestamps: true }`.
- **`__v`**: a version number Mongoose uses internally. You can ignore it.

SQL vs NoSQL in one minute

MongoDB is a **NoSQL** database: documents in a collection don't have to share the same shape, and there are no foreign keys. **SQL** databases (PostgreSQL, MySQL) use strict tables with fixed columns, and the database itself enforces links between tables. Mongoose brings back some of that structure through schemas.

Schema and model

- A **schema** is the blueprint: which fields a document has, their types and their rules.
- A **model** is the tool made from the schema. You use it to create, find, update and delete documents.

```
const patientSchema = new mongoose.Schema({ name: String, age: Number });
// blueprint
const Patient = mongoose.model('Patient', patientSchema);                  //
tool
```

Mongoose turns the model name into the collection name: `Patient` → `patients`.

The model methods we use

Code	What it does
`Patient.create({...})`	Insert a new document
`Patient.find({ createdBy: userId })`	Get all documents that match
`Patient.findOne({ _id: id, createdBy: userId })`	Get the first match, or `null`
`Patient.findById(id)`	Get one by `_id`
`Patient.exists({...})`	Is there at least one match?
`Patient.findOneAndUpdate(filter, changes, { new: true })`	Update one and return the new version
`doc.deleteOne()` / `Mapping.deleteMany({...})`	Delete one / delete all matches
`.sort({ createdAt: -1 })`	Order results (-1 = newest first)
`.populate('doctorId')`	Replace an id with the full document it points to (section 8)
`.select('+password')`	Also return a field that is hidden by default

`id` vs `_id`

The frontend uses `id`, not `_id`. One line at the top of `models/index.js` makes every response also include `id`:

```
mongoose.set('toJSON', { virtuals: true });
```

A **virtual** is a field that is calculated instead of stored. Mongoose gives every document an `id` virtual (the `_id` as text). This line must run before the models are created, which is why it sits at the top of `models/index.js`. When it was removed once, every `id` disappeared and assigning doctors broke with "not a valid id".

7. The four models, field by field

Our app stores four kinds of things, so it has four models, one file each in `src/models/`. Each one is explained below with its code and what every option means.

Field options you will see

Option	Meaning
`type: String` / `Number`	The kind of value stored
`required: true`	Saving fails if this field is missing
`unique: true`	No two documents may have the same value (creates a unique **index**, section 9)
`trim: true`	Removes spaces at the start and end: `" Asha "` → `"Asha"`
`lowercase: true`	Stores the text in small letters, so `Asha@X.com` and `asha@x.com` count as the same email
`enum: [...]`	Only these values are allowed
`default: 0`	The value used when none is given
`select: false`	Hidden from query results unless asked for
`ref: 'User'`	This field holds the `_id` of a document in another model
`phone: String`	Short way of writing `{ type: String }` with no other rules

User (`models/User.js`) – people who log in

```
const userSchema = new mongoose.Schema({
  name: { type: String, required: true, trim: true },
  email: { type: String, required: true, unique: true, trim: true, lowercase:
true },
  password: { type: String, required: true, select: false },
}, { timestamps: true });

userSchema.pre('save', async function () {
  if (this.isModified('password')) this.password = await
bcrypt.hash(this.password, 10);
});
```

- **`email` is unique**, so two accounts can't share an email.
- **`password` has `select: false`**, so normal queries never return it. Login asks for it on purpose with `.select('+password')`.

- **`pre('save')`** is a **hook**: a function Mongoose runs automatically just before a document is saved. Here it replaces the plain password with a **hash** (section 11). `isModified('password')` makes sure we only hash when the password changed, not on every save.

Patient (`models/Patient.js`) – private to the user who added them

```
const patientSchema = new mongoose.Schema({
  name: { type: String, required: true, trim: true },
  age: { type: Number, required: true },
  gender: { type: String, required: true, enum: ['male', 'female', 'other']
},
  phone: String,
  email: { type: String, trim: true, lowercase: true },
  address: String,
  medicalHistory: String,
  createdBy: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required:
true },
}, { timestamps: true });
```

- **`gender`** uses **`enum`**, so only `male`, `female` or `other` can be saved.
- **Patient email is not unique**: two patients (say, family members) may share one email.
- **`createdBy`** stores the `_id` of the user who added the patient. Every patient query filters by it, e.g. `Patient.find({ createdBy: req.user._id })`. That single filter is what keeps each user's patients private.

Doctor (`models/Doctor.js`) – shared by everyone

```
const doctorSchema = new mongoose.Schema({
  name: { type: String, required: true, trim: true },
  specialization: { type: String, required: true, trim: true },
  email: { type: String, required: true, unique: true, trim: true, lowercase:
true },
  phone: String,
  experienceYears: { type: Number, default: 0 },
  createdBy: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required:
true },
}, { timestamps: true });
```

- **Doctor `email` is unique**, so the same doctor can't be added twice.
- **Doctors are not filtered by `createdBy` when listing**, so everyone sees all doctors.

- **`createdBy` is still used for permission:** the controller checks `doctor.createdBy.equals(req.user._id)` before an edit or delete, and replies 403 if it doesn't match. (`.equals()` is needed because ObjectIds are objects, and `===` would compare two different objects.)

Mapping (`models/Mapping.js`) – the link between a patient and a doctor

```
const mappingSchema = new mongoose.Schema({
  patientId: { type: mongoose.Schema.Types.ObjectId, ref: 'Patient',
required: true },
  doctorId: { type: mongoose.Schema.Types.ObjectId, ref: 'Doctor', required:
true },
  notes: String,
  createdBy: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required:
true },
}, { timestamps: true, collection: 'patient_doctor_mappings' });

mappingSchema.index({ patientId: 1, doctorId: 1 }, { unique: true, name:
'unique_patient_doctor' });
```

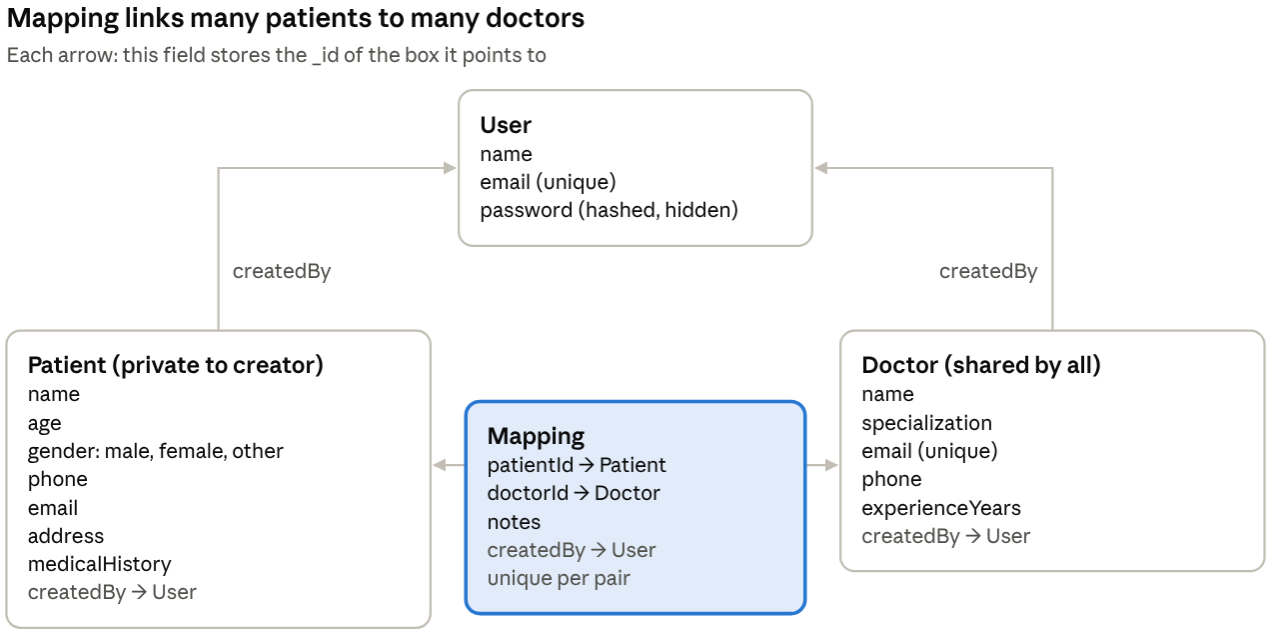
- **Each mapping document is one assignment:** "this doctor looks after this patient". Three doctors for one patient = three mapping documents.
- **`collection: 'patient_doctor_mappings'`** sets the collection name ourselves instead of the default `mappings`.
- **The index line** makes the pair (patient, doctor) unique: the same doctor can't be assigned to the same patient twice, but the patient can have any number of different doctors. Section 9 explains indexes.
- **`notes`** is optional text about the assignment, e.g. "follow-up for blood pressure".

`models/index.js`

This file loads all four models in one place, so other files can write `const { Patient, Doctor } = require('../models');`. It also holds the `toJSON` line from section 6.

8. How the models are linked

Models are linked by storing another document's `_id` in a field. There are two kinds of link in this app.



data model · 4 models, 4 links

Patients and doctors each point to the user who created them; each Mapping points to one patient and one doctor.

One-to-many: User → Patients, User → Doctors

One user can add many patients, but each patient has exactly one creator. We store the user's `_id` on the patient (`createdBy`). The "many" side holds the id of the "one" side; we never keep a list of patient ids inside the user.

Many-to-many: Patients ↔ Doctors

One patient can have many doctors, and one doctor can have many patients. Storing a list on either side gets messy (two lists to keep in sync, lists that grow forever). The usual answer is a third collection that holds one document per link. That is our **Mapping** model, also called a **join collection** (in SQL, a **join table**).

Mapping `_id`	patientId	doctorId
m1	Asha	Dr Rao
m2	Asha	Dr Mehta
m3	Ravi	Dr Rao

Asha has two doctors (m1, m2) and Dr Rao has two patients (m1, m3). (The table shows names to keep it readable; the real documents store ids.)

`populate` : swapping an id for the real document

A mapping only stores ids. To show names, we ask Mongoose to fetch the linked documents:

```
const mappings = await Mapping.find({ patientId: patient._id
}).populate('doctorId');
// before populate: m.doctorId = '65f1...'
// after populate:  m.doctorId = { id: '65f1...', name: 'Dr Rao',
specialization: 'Cardiology', ... }
const doctors = mappings.map((m) => m.doctorId);
```

`populate` works because the schema says `ref: 'Doctor'`. Behind the scenes, Mongoose runs a second query: "find doctors whose `_id` is in this list". This is MongoDB's version of a **join**.

The mapping list uses `.populate('patientId doctorId')` to fill both sides at once. That's why the frontend reads `m.patientId.name` and `m.doctorId.name`.

Deleting linked data (cascade)

If you delete a patient, their mapping documents would point to a patient that no longer exists. SQL databases can delete those automatically (`ON DELETE CASCADE`), but MongoDB has no foreign keys, so our controllers do it by hand:

```
await Mapping.deleteMany({ patientId: patient._id }); // remove this
patient's assignments first
await patient.deleteOne();
```

Deleting a doctor does the same with `doctorId`.

9. Indexes

An index is like the index at the back of a book: instead of reading every page to find "Mongoose", you look it up and jump straight to page 42. Without an index, MongoDB checks every document in the collection one by one (a **collection scan**).

Indexes in our app

Index	Where it comes from	Why
`_id` on every collection	MongoDB adds it automatically	Fast lookup by id. This is why `populate` needs no extra index

Index	Where it comes from	Why
`email` on users	`unique: true` in the User schema	No two accounts with the same email
`email` on doctors	`unique: true` in the Doctor schema	No duplicate doctors
`{ patientId, doctorId }` on mappings	`mappingSchema.index(...)`	The same pair can't be saved twice; also speeds up "find mappings for this patient"

Two jobs an index can do

1. **Speed:** find documents by a field without scanning everything. The compound index starts with `patientId`, so `Mapping.find({ patientId })` can use it too.
2. **A rule (`unique`):** MongoDB itself refuses a second document with the same value. Even if two requests arrive at the same moment, only one can win. The loser gets a **duplicate key error**, code `11000`, which our error handler turns into a 409 reply.

Do joins need indexes?

No. `populate` looks up documents by `_id`, which is always indexed. You add your own index when you need a unique rule, or when you often filter or sort by a field on a large collection. With a few hundred records you won't notice the difference, but with millions you will.

The cost

Indexes aren't free. Each one takes disk space and makes every insert and update a little slower, because the index must be updated too. So add them for a reason, not "just in case".

Who creates them?

Mongoose builds the indexes described in your schemas automatically when the app starts (a feature called `autoIndex`). If you rename an index in code, the old one stays in the database until you remove it, for example in MongoDB Atlas under the collection's **Indexes** tab.

10. Validation and error handling

Never trust what the client sends. Anyone can send any request with tools like Postman, skipping the checks in your React forms. So the backend checks everything again.

Step 1: validators (`src/validators/index.js`)

We use the **express-validator** library. Each rule checks one field:

```
body('age').isInt({ min: 0, max: 150 }).withMessage('Age must be between 0
and 150').toInt(),
body('gender').isIn(['male', 'female', 'other']).withMessage('Gender must be
male, female or other'),
param('id').isMongoId().withMessage('id is not a valid id'),
```

- `body('age')` checks `req.body.age`. `param('id')` checks the `:id` in the URL.
- `isMongoId()` makes sure an id has the 24-character ObjectId shape before we even ask the database.
- `.toInt()` turns `"34"` (text) into `34` (a number).

Step 2: the `validate` middleware

It collects all failed rules. If there are any, it stops the request with a 400 and a list of problems, so the controller never runs.

Step 3: `ApiError` for expected problems

Inside controllers we throw errors that carry a status code:

```
if (!patient) throw ApiError.notFound('Patient not found'); // becomes a 404
```

`ApiError` is a tiny class: a normal JavaScript `Error` plus a `statusCode`, with shortcuts like `badRequest` (400), `unauthorized` (401), `forbidden` (403), `notFound` (404) and `conflict` (409).

Step 4: the error handler (`middleware/errorHandler.js`)

All errors end up in one function at the end of `app.js`. It picks the status code and sends one consistent JSON reply:

- An `ApiError` → its own status and message.
- A MongoDB duplicate key error (code `11000`) → 409, with "This doctor is already assigned to this patient" or "This email is already in use".

- Anything unexpected → 500 "Something went wrong". The real error is printed in the server terminal, but hidden from the user, because error details can reveal how your system works.

There's also a `notFound` handler for URLs that don't exist, e.g. `GET /api/bananas` → 404.

Why `404` instead of `403` for someone else's patient?

If user B asks for user A's patient, we reply "Patient not found", not "Forbidden". A 403 would confirm that the patient exists, which already leaks information. Doctors are different: everyone can see them anyway, so editing someone else's doctor gives a clear 403.

11. Passwords: hashing with bcrypt

We never store a password as typed. If the database ever leaked, every user's password (often reused on other sites) would be exposed. Instead we store a **hash**.

What a hash is

A hash is a one-way scramble. `hash("pass1234")` always gives the same kind of result, but you can't turn the result back into `pass1234`.

```
pass1234  →  $2a$10$N9qo8uLOickgx2ZMRZoMye...
```

How login works without knowing the password

1. **Register:** the `pre('save')` hook in the User model runs `bcrypt.hash(password, 10)` and stores only the hash.
2. **Login:** we load the stored hash and run `bcrypt.compare(typedPassword, user.password)`. bcrypt hashes the typed password the same way and checks whether the results match.

Why bcrypt and not a simple hash

- **Salt:** bcrypt adds random characters before hashing, so two users with the same password get different hashes. An attacker can't use a ready-made list of known hashes.
- **Slow on purpose:** the `10` is the **cost factor**: how many rounds of work to do (2 to the power of 10). Logging in once is still fast for a real user, but guessing millions of passwords becomes very slow for an attacker.

Two habits that protect users

- **The hash is never sent back.** `select: false` hides it from queries, and the auth controller only returns `{ id, name, email }` via the `publicUser()` helper.
- **The same error for both mistakes.** Wrong email and wrong password both get "Invalid email or password". Saying "email not found" would tell an attacker which emails have accounts.

12. Staying logged in: sessions vs tokens

HTTP is **stateless**: the server forgets you the moment it replies. Logging in proves who you are once, so every later request needs some proof again. There are two common ways to give that proof.

Option A: sessions (the classic way)

Like a cloakroom ticket.

1. You log in. The server creates a **session**, a small record such as `{ sessionId: 'abc123', userId: '65f1...' }`, and stores it in memory, Redis or the database.
2. The server sends the browser a **cookie** holding only `abc123`.
3. The browser sends that cookie with every request automatically.
4. The server looks up `abc123` in its session store to find out who you are.
5. Logout = the server deletes the session, and the cookie stops working at once.

In Express this is done with libraries like `express-session`.

Option B: tokens (what our app uses)

Like a signed visitor badge.

1. You log in. The server creates a **token** that already contains your user id and signs it with a secret key.
2. The browser stores the token (our app uses `localStorage`) and sends it in the `Authorization` header.
3. The server checks the signature. If it's valid, it trusts the user id inside. No lookup in a session store is needed.

Side by side

	Sessions	Tokens (JWT)
What the browser holds	A random session id (in a cookie)	The signed token itself
What the server stores	Every active session	Nothing (just the secret key)
Logout	Delete the session: instant	The token works until it expires, unless you keep a block list
Many servers	All servers must share the session store	Any server with the secret can check a token
Good for	Classic websites, server-rendered pages	APIs, single-page apps, mobile apps
Main risk	Cookie theft (protect with `httpOnly`, `secure`, `sameSite`)	Token theft, e.g. through an XSS bug reading `localStorage`

Cookies vs localStorage

- A **cookie** is sent by the browser automatically. Marked `httpOnly`, JavaScript can't read it, which protects it from XSS attacks.
- **localStorage** is a small storage box in the browser that JavaScript can read. Simple, but any malicious script on the page could read the token too.

Both approaches are valid. Many production apps combine them: a JWT stored in an `httpOnly` cookie.

13. JWT: access and refresh tokens

Our app gives every user two tokens at login: a short-lived **access token** used on every request, and a long-lived **refresh token** used only to get a new access token.

What a JWT looks like

A **JWT** (JSON Web Token, said "jot") is three pieces joined by dots: `header.payload.signature`.

- **Header:** which signing method is used.
- **Payload:** the data, e.g. `{ sub: '65f1...', type: 'access', exp: 1792140000 }`. `sub` = the user id, `exp` = the expiry time.

- **Signature:** made from the header, the payload and a secret key that only the server knows.

Anyone can decode and read the payload (it isn't encrypted), so never put passwords or private data in it. But nobody can change it without the secret: if one character changes, the signature no longer matches and the server rejects it.

When each token is created (`utils/tokens.js`)

```
const signAccess  = (user) => jwt.sign({ sub: user.id, email: user.email,
type: 'access' }, env.jwt.accessSecret,  { expiresIn: '1h' });
const signRefresh = (user) => jwt.sign({ sub: user.id, type: 'refresh' },
env.jwt.refreshSecret, { expiresIn: '7d' });
```

Moment	Access token	Refresh token
Register	Created	Created
Login	Created	Created
`POST /auth/refresh`	A new one is created	Not created; the old one is reused

When each token is checked

Token	Checked in	With
Access	`middleware/auth.js` , on every protected route	`JWT_ACCESS_SECRET`
Refresh	`authController.refresh` , only on `/auth/refresh`	`JWT_REFRESH_SECRET`

Using two different secrets means a refresh token can't be used as an access token, or the other way round.

The full flow

1. **Login:** the server returns `{ access, refresh }` , and the frontend saves both.
2. **Normal request:** the frontend sends `Authorization: Bearer <access>` . The auth middleware runs `jwt.verify` , loads the user and puts it on `req.user` .
3. **After 1 hour:** the access token has expired, so the server replies 401 "Access token expired".

4. **Silent refresh:** `frontend/src/api.js` notices the 401, calls `POST /api/auth/refresh` with the refresh token, saves the new access token and retries the original request. The user notices nothing.
5. **After 7 days:** the refresh token has expired too. The refresh call fails, the frontend logs the user out, and they log in again.

Why two tokens instead of one

- One token lasting 7 days: simple, but a stolen token works for 7 days.
- One token lasting 1 hour: safe, but users must log in every hour.
- Two tokens: the one sent on every request (most likely to leak) expires in 1 hour, and users still stay logged in for 7 days.

Known limits of our simple version

- **No real logout on the server:** tokens aren't stored, so they can't be cancelled. They stay valid until they expire. A production app would store refresh tokens in the database so it can delete them.
- **No rotation:** a production app usually issues a new refresh token on every refresh and blocks the old one.

14. Environment variables, ports and CORS

These three topics cause most "it doesn't run on my machine" problems, so they're worth knowing well.

Environment variables (`.env`)

Some settings differ between your laptop and the live server, and some are secret. So they don't go in the code; they go in a `.env` file:

```
PORT=5001
CLIENT_URL=http://localhost:5173
MONGODB_URI=mongodb+srv://user:password@cluster.mongodb.net/healthcare_db
JWT_ACCESS_SECRET=a-long-random-string
JWT_REFRESH_SECRET=another-long-random-string
```

- The `dotenv` library loads this file into `process.env` when the app starts.
- `config/env.js` reads the values in one place and stops the app with a clear message if a required one is missing.
- `.env` is listed in `.gitignore`, so it never goes to GitHub. `.env.example` is the safe copy with fake values, showing teammates what to fill in.

- **Frontend env variables are not secret.** Anything in `frontend/.env` (like `VITE_API_URL`) ends up in the code the browser downloads.

Ports

A port is like a door number on a computer. One computer runs many programs, and each listens on its own door: the API on 5001, the React dev server on 5173, a local MongoDB on 27017.

`EADDRINUSE` error: "address already in use" means another program is already on that port. On a Mac, AirPlay Receiver uses port 5000, which is why this project moved to 5001. To see what is using a port, run `lsof -i :5001`.

CORS

Browsers protect users with the **same-origin policy**: a page from one **origin** (protocol + host + port) can't freely read responses from another origin. `http://localhost:5173` and `http://localhost:5001` are different origins, because the ports differ.

CORS (Cross-Origin Resource Sharing) is how a server says "I allow requests from that origin". Our backend has one line:

```
app.use(cors({ origin: env.clientUrl })); // allow only our frontend
```

Two important facts:

- CORS is enforced by the **browser**, not the server. Postman and curl ignore it completely.
- In development we mostly avoid CORS altogether, because the Vite proxy makes the frontend and API look like one origin (section 2).

A real bug from this project: `frontend/.env` had `VITE_API_URL=http://localhost:5002`, the wrong port. The browser called a server that wasn't running and showed `ERR_CONNECTION_REFUSED`. Removing that line let the Vite proxy do its job. Remember that Vite reads `.env` only at startup, so restart it after a change.

15. Things we removed to keep the code simple

We kept this project small on purpose so it's easy to learn from. Real production apps usually add the pieces below. You don't need them to understand the app, but you should know they exist and why.

Removed	What it does	Why you might add it back
Helmet (`helmet` package)	Adds security headers to every response, e.g. telling browsers not to load your site inside another site's frame, and to use HTTPS	Any app going live on the internet. One line: `app.use(helmet())`
asyncHandler	Wrapped each async route so its errors reached the error handler	Only needed on Express 4. Express 5 does this itself
toJSON transform	Removed `_id`, `__v` and `password` from every response automatically	When you want clean responses everywhere without remembering to strip fields by hand
Stricter CORS	Allowed a list of origins, any localhost port in development, and named the allowed headers	When your frontend runs on several domains (staging, production)
Schema validation (`maxlength`, `min`/`max`, email `match`)	The database layer double-checks values, not just the request validators	When data can be written from more than one place (scripts, admin tools, other services)
Manual duplicate checks	Checked "does this email exist?" before saving, for friendlier messages	Now the unique index plus the error handler cover it
Request size limit	`express.json({ limit: '100kb' })` refused very large bodies	To protect the server from huge requests
Graceful shutdown	Closed the database connection cleanly when the server stopped	Production servers that restart often (deployments)
Explicit `m.init()`	Waited for indexes to be built before accepting requests	So the unique rules definitely exist from the very first request

Other high-level ideas worth knowing

- **Rate limiting** (e.g. `express-rate-limit`): allow only N requests per minute from one address. It stops password-guessing on `/login`.
- **HTTPS:** encrypts traffic between browser and server, so tokens can't be read in transit. Always on for live sites.
- **Logging and monitoring:** `morgan` prints each request in the terminal. Production apps send logs to a service so problems can be found later.
- **Pagination:** `GET /patients` returns everything. With thousands of rows you would return pages, e.g. `?page=2&limit=20` using `.skip()` and `.limit()`.
- **Roles:** right now every user has the same permissions. Real clinics need roles such as admin, doctor and receptionist.
- **Transactions:** deleting a patient and their mappings is two separate steps. If the server crashed in between, the data could be left half-done. MongoDB transactions can make several steps all-or-nothing.
- **Automated tests:** scripts that call the API and check the answers, so you know a change didn't break anything.
- **Health check:** `GET /api/health` replies `{ status: 'ok' }`, so a hosting platform can check that the server is alive.

16. Glossary and practice

Glossary

Word	Meaning in one line
API	The set of URLs a backend offers for other programs to call
Backend	The server program that holds the rules and talks to the database
bcrypt	A library for hashing passwords slowly and with a salt
Collection	A group of documents in MongoDB, like a table
Controller	The function that does the work for one route and sends the reply
CORS	Browser rules about which other websites may call your API

Word	Meaning in one line
Document	One record in MongoDB, stored like a JSON object
Environment variable	A setting read from `.env` instead of written in code
Express	A Node.js library for building web servers
Frontend	The part users see in the browser (React here)
Hash	A one-way scramble of a value, used for passwords
Index	A lookup structure that makes searches fast or enforces uniqueness
JSON	A text format for data: `{ "key": "value" }`
JWT	A signed token that proves who the user is
Join collection	A collection whose documents link two others (our Mapping)
Middleware	A function that runs before the controller and can stop the request
Model	The Mongoose tool for reading and writing one collection
Mongoose	A library that adds schemas and handy methods on top of MongoDB
ObjectId	MongoDB's 24-character unique id type
populate	Replaces a stored id with the full document it points to
Port	The "door number" a program listens on
Refresh token	A long-lived token used only to get a new access token
Route	A method + URL connected to functions
Schema	The blueprint of a document: fields, types, rules
Session	Login state stored on the server, found through a cookie
Stateless	The server doesn't remember earlier requests

Word	Meaning in one line
Status code	The number on a response saying how it went (200, 404…)
Validation	Checking that input is correct before using it
Virtual	A field that is calculated, not stored (like `id`)

Practice exercises

Try these in order. Each one touches a different part of the guide.

- [ ] Register two users in Postman. With user B's token, ask for user A's patient. Which status code do you get, and why? (Section 10)
- [ ] Find the `users` collection in MongoDB Atlas and look at a password. What do you see? (Section 11)
- [ ] Paste your access token into jwt.io and read the payload. Can you find your user id and the expiry time? (Section 13)
- [ ] Assign the same doctor to the same patient twice. Which line of code stops it? (Section 9)
- [ ] Add a `bloodGroup` field to the Patient model, with an `enum` of the 8 blood groups and a matching validator. (Sections 7 and 10)
- [ ] Comment out `mongoose.set('toJSON', { virtuals: true })`, restart, and look at a patient response. What's missing, and what breaks in the UI? (Section 6)
- [ ] Add a new route `GET /api/doctors/:id/patients` that lists one doctor's patients using the Mapping model and `populate`. (Sections 5 and 8)
- [ ] Change the access token lifetime to `1m`, wait a minute, and use the app. Watch the Network tab in the browser: can you see the silent refresh? (Section 13)